

Structured Synchronous Reactive Programming for Game Development --- Case Study: On Rewriting Pingus from C++ to Céu

Introduction

Pingus [1] is an open-source [2] clone of Lemmings [3], a puzzle-platformer video game.

The objective of the game is to guide a group of penguins through a number of obstacles towards a designated exit [4].

Pingus is developed in standard object-oriented C++, the *lingua franca* of game development [5]. The codebase is about 40.000 lines of code (LoCs) [6], divided into the engine, level editor, auxiliary libraries, and the game logic itself.

Céu [7,8] is a programming language that aims to offer a concurrent and expressive alternative to C/C++ with the characteristics that follow:

- **Reactive:** code only executes in reactions to events.
- **Structured:** programs use structured control mechanisms, such as `await` (to suspend a line of execution), and `par` (to combine multiple lines of execution).
- **Synchronous:** reactions run atomically and to completion on each line of execution, i.e., there's no implicit preemption or real parallelism.



Figure 1: Pingus gameplay

Structured programming eliminates the *callback hell* [9], letting programmers write code in [direct/sequential style](#). Céu supports [logical parallelism](#) with a resource-efficient implementation in terms of memory and CPU usage. [The runtime is single threaded and the language requires no garbage collection.](#)

According to Tim Sweeney (of Unreal Engine fame), about half the complexity in game development resides in *simulation* (aka *game logic*), but which accounts for only 10% of the CPU budget [10]. The game logic "models the state of the game world as interacting objects evolve over time". The high development costs contrasting with the low impact on performance appeals for alternatives with productivity in mind, especially considering that it is the game logic that varies the most between projects. Sweeney states that "will gladly sacrifice 10% of our performance for 10% higher productivity".

	Game Simulation	Numeric Computation	Shading
Languages	C++, Scripting	C++	CG, HLSL
CPU Budget	10%	90%	n/a
Lines of Code	250,000	250,000	10,000
FPU Usage	0.5 GFLOPS	5 GFLOPS	500 GFLOPS

Figure 2: Three kinds of code

The main motivation for rewriting Pingus to Céu is to suggest structured synchronous reactive programming as an expressive and productive alternative for game logic development. In Pingus, the game logic also accounts for almost half the size of the codebase (18.173 from 39.362 LoCs, or 46%).

The rewriting process consisted of [identifying sets of callbacks implementing control-flow behaviors](#) in the game and translating them to Céu using appropriate structured constructs. As an example, a double mouse click is characterized by a first click, followed by a maximum amount of time, followed by a second click. This behavior depends on different events (clicks and timers) which have to occur in a particular order. In C++, the implementation involves callbacks crossing reactions to successive events which manipulate state variables explicitly.

[We can identify control-flow behaviors in C++ by looking for class members with identifiers resembling verbs, statuses, and counters \(e.g., `pressed`, `particle\_thrown`, `mode`, and `delay\_count`\).](#) Good chances are that variables with these "suspicious names" encode some form of control-flow progression that cross multiple callback invocations.

We rewrote 126 of the 272 files (46%) which account for 9.186 of the 18.173 LoCs (51%) comprising the game logic of Pingus [11]. Half of the game logic relates to non-reactive code, such as configurations and options, saved games and serialization, maps and levels descriptions, string formatting, collision detection, graph algorithms, etc. This part remains unchanged and relies on the seamless integration between Céu and C/C++ to remain usable. From the 9.186 touched LoCs, we removed all headers, declarations, trivial getters & setters, and other innocuous statements, resulting in 70 implementation files with 4.135 dense LoCs originally written in C++ [12]. We did the same with the implementation in Céu, resulting in 3.697 dense LoCs [13]. The table that follows summarizes the condensed codebase in the two implementations:

Path	Céu	C++	Céu/C++	Description
game/	2064	2268	0.91	the main gameplay
./	710	679	1.05	main functionality
objs/	470	478	0.98	world objects (tiles, traps, etc)
pingu/	884	1111	0.80	pingu behaviors
./	343	458	0.75	main functionality
actions/	541	653	0.83	pingu actions (bomber, climber, etc)
worldmap/	468	493	0.95	campaign worldmap
screens/	1109	1328	0.84	menus and screens
option/	347	357	0.97	option menu
others/	762	971	0.78	other menus and screens
misc/	56	46	1.22	miscellaneous functionality
	3697	4135	0.89	

This report focuses on a qualitative analysis for the programming techniques that we applied during the rewriting process. Not all techniques result in reduction in LoCs (especially considering the verbose syntax of Céu), but have other effects such as eliminating shared variables and dependencies between classes. Nonetheless, the lowest ratio numbers above correlate to the parts of the game logic that we consider more susceptible to structured reactive programming. For instance, the *Pingu* behavior (*ratio 0.80*) contains complex animations that are affected by timers, game rules, and user interaction. In contrast, the *Option* screen (*ratio 0.97*) is a simple UI grid with trivial mouse interactions.

We selected 9 representative game behaviors and describe their implementations in C++ and Céu. We also categorized these examples in 5 abstract C++ control-flow patterns that likely apply to other games:

1. **Finite State Machines:** State machines describe the behavior of entities by mapping event occurrences to transitions between states that trigger appropriate actions.
 - [[case 1](#) | [case 2](#) | [summary](#)]
2. **Continuation Passing:** The completion of a long-lasting activity may carry a continuation, i.e., some action to execute next.
 - [[case 1](#) | [case 2](#) | [summary](#)]
3. **Dispatching Hierarchies:** Entities typically form a dispatching hierarchy in which a container that receives a stimulus automatically forwards it to its managed children.
 - [[case 1](#) | [summary](#)]
4. **Lifespan Hierarchies:** Entities typically form a lifespan hierarchy in which a terminating container entity automatically destroys its managed children.
 - [[case 1](#) | [case 2](#) | [summary](#)]
5. **Signaling Mechanisms:** Entities often need to communicate explicitly through signaling mechanisms, especially if there is no hierarchy relationship between them.
 - [[case 1](#) | [case 2](#) | [summary](#)]

Conclusion

Author

1) Finite State Machines

State machines describe the behavior of entities by mapping event occurrences to transitions between states that trigger appropriate actions.

1.1) The *Armageddon* Button Double Click

In Pingus, a double click in the *Armageddon* button at the bottom right of the screen literally explodes all pingus (Figure 3).

C++

In C++, the class `ArmageddonButton` [9] implements methods for rendering the button and handling mouse and timer events. In the code that follows, we focus on the double click detection, hiding unrelated parts with `<...>`:

```

1: ArmageddonButton::ArmageddonButton(<...>):
2:     RectComponent(<...>),
3:     pressed(false); // button state: initially not pressed
4:     press_time(0); // how long since the 1st click?
5:     <...>
6: {
7:     <...>
8: }
```

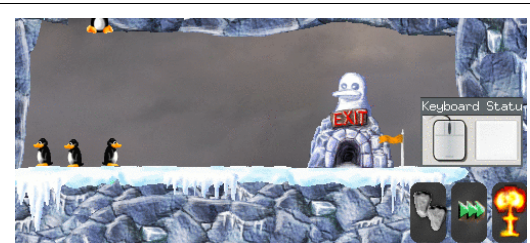


Figure 3: Double click detection

```

9:
10: void ArmageddonButton::draw (<...>) {
11:     <...>
12: }
13:
14: void ArmageddonButton::update (float delta) {
15:     <...>
16:     if (pressed) {
17:         press_time += delta;
18:         if (press_time > 1.0f) {
19:             pressed = false;    // giving up, 1st click was
20:             press_time = 0;      // too long ago
21:         }
22:     } else {
23:         <...>
24:         press_time = 0;
25:     }
26: }
27:
28: void ArmageddonButton::on_click (<...>) {
29:     if (pressed) {
30:         server->send_armageddon_event();
31:     } else {
32:         pressed = true;
33:     }
34: }

```

The methods `update` (ln. 14-26) and `on_click` (ln. 28-34) are examples of *short-lived callbacks*, which are pieces of code that execute atomically in reaction to external input events. The callback `on_click` reacts to mouse clicks detected by the button base class `RectComponent` (ln. 2), while the callback `update` continuously reacts to the passage of time, frame by frame. Callbacks are short lived because they must react to input as fast as possible to let other callbacks execute, keeping the game with real-time responsiveness.

The class first initializes the variable `pressed` to track the first click (ln. 3,32). It also initializes the variable `press_time` to count the time since the first click (ln. 4,17). If another click occurs within 1 second, the class signals the double click to the application (ln. 30). Otherwise, the `pressed` and `press_time` state variables are reset (ln. 19-20).

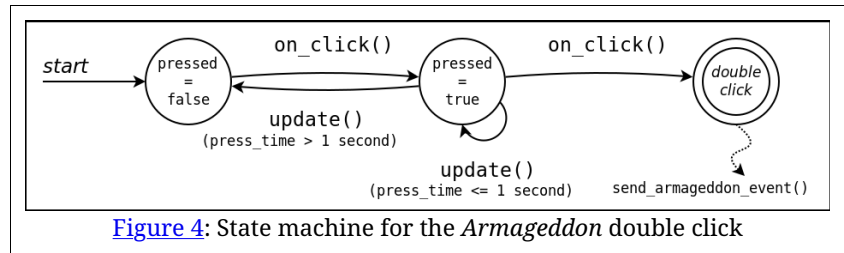


Figure 4: State machine for the *Armageddon* double click

Figure 4 illustrates how we can model the double-click behavior in C++ as a state machine. The circles represent the state of the variables in the class, while the arrows represent the callbacks manipulating state.

Note in the code how the accesses to the state variables are spread across the entire class. For instance, the distance between the initialization of `pressed` (ln. 3) and the last access to it (ln. 32) is over 40 lines in the original file [9]. Arguably, this dispersion of code across methods makes the understanding and maintenance of the double-click behavior more difficult. Also, even though the state variables are private, unrelated methods such as `draw`, which is defined in middle of the class (ln. 10-12), can potentially access them.

Céu

Céu provides structured constructs to deal with events, aiming to eradicate explicit manipulation of state variables for control-flow purposes. The equivalent code in Céu for the double-click detection is as follows [9]:

```

1: do
2:   var& RectComponent c = <...>;    // (the symbol `&' denotes an alias declaration)
3:   <...>
4:   loop do
5:     await c.component.on_click;
6:     watching 1s do
7:       await c.component.on_click;
8:       break;
9:     end
10:  end
11:  <...>
12:  emit outer.game.go_armageddon;
13: end

```



The loop detection (ln. 4-10) awaits the first click (ln. 5) and then, while *watching* 1 second (ln. 6-9), awaits the second click (ln. 7). If the second click occurs within 1 second, the `break` terminates the loop (ln. 8) and the `emit` signals the double click to the application (ln. 12). Otherwise, the `watching` block as a whole aborts and restarts the loop, falling back to the first click `await` (ln. 5).

Double click detection in Céu doesn't require state variables and is entirely self-contained in the `loop` body (ln. 4-10).

Furthermore, these 7 lines of code **only** detect the double click, leaving the actual effect to happen outside the loop (ln. 12).

1.2) The *Bomber* Action

The *Bomber* action explodes the clicked pingu, throwing particles around and also destroying the terrain under its radius (Figure 5).

We can model the explosion animation with a sequential state machine (Figure 6) with actions associated to specific frames as follows:

1. 0th frame: plays a "Oh no!" sound.
2. 10th frame: plays a "Bomb!" sound.
3. 13th frame: throws particles, destroys the terrain, and shows an explosion sprite.
4. Game tick: hides the explosion sprite.
5. Last frame: kills the pingu.

([This video](#) presents the sound effects.)

C++

In C++, the class `Bomber` [9] defines the callbacks `draw` and `update` to manage the state machine described above:

```

1: Bomber::Bomber (Pingu* p) :
2:   <...>
3:   sprite(<...>),           // bomber sprite
4:   sound_played(false),    // tracks state 2
5:   particle_thrown(false), // tracks state 3
6:   colmap_exploded(false), // tracks state 3
7:   gfx_exploded(false)     // tracks state 4
8: {
9:   <...>
10:  // 1. 0th frame: plays a "Oh no!" sound.
11:  get_world()->play_sound("ohno", pingu->get_pos());
12: }
13:
14: void Bomber::update () {
15:   sprite.update();
16:   <...> // pingu movement
17:
18:   // 2. 10th frame: plays a "Bomb!" sound.
19:   if (sprite.get_current_frame()==10 && !sound_played) {
20:     sound_played = true;
21:     get_world()->play_sound("plop", pingu->get_pos());
22:   }
23:
24:   // 3. 13th frame: throws particles, destroys the terrain, shows an explosion sprite
25:   if (sprite.get_current_frame()==13 && !particle_thrown) {
26:     particle_thrown = true;
27:     get_world()->get_pingu_particle_holder()->add_particle(...);
28:   }
29:   if (sprite.get_current_frame()==13 && !colmap_exploded) {
30:     colmap_exploded = true;
31:     get_world()->remove(bomber_radius, <...>);
32:   }
33:
34:   // 5. Last frame: kills the Pingu
35:   if (sprite.is_finished ()) {
36:     pingu->set_status(Pingu::PS_DEAD);
37:   }
38: }
39:
40: void Bomber::draw (SceneContext& gc) {
41:   // 3. 13th frame: throws particles, destroys the terrain, shows an explosion sprite
42:   // 4. Game tick: hides the explosion sprite
43:   if (sprite.get_current_frame()==13 && !gfx_exploded) {
44:     gfx_exploded = true;
45:     gc.color().draw(explo_surf, <...>);
46:   }
47:   gc.color().draw(sprite, pingu->get_pos());
48: }

```



Figure 5: The *Bomber* action

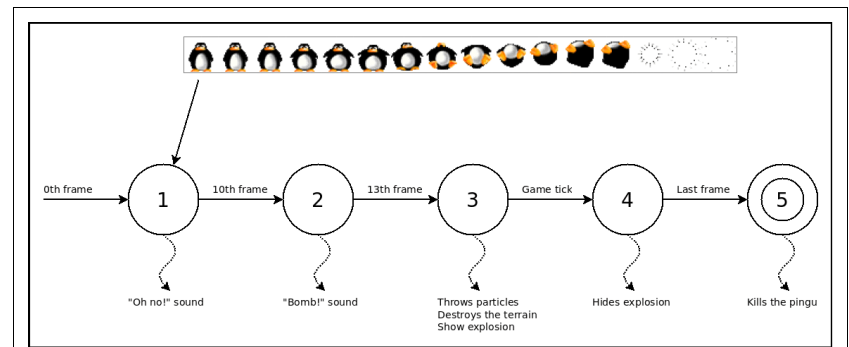


Figure 6: State machine for the *Bomber* animation

The class first defines one state variable for each action to perform (ln. 4-7). The "Oh no!" sound plays as soon as the object starts in *state-1* (ln. 11). The `update` callback (ln. 14-38) updates the pingu animation and movement on every frame regardless of its current state (ln. 15-16). When the animation reaches the 10th frame, it plays the "Bomb!" sound and switches to *state-2* (ln. 18-22). The state variable `sound_played` is required because the sprite frame doesn't necessarily advance on every update invocation (e.g., `update` may execute twice during the 10th frame). The same reasoning and technique applies

to *state-3* (ln. 25-32 and 43-44). The explosion sprite appears in a single frame in *state-4* (ln. 45). Finally, the pingu dies after the animation frames terminate (ln. 35-37).

Note that a single numeric state variable suffices to track the states, but the original authors probably chose to encode each state in an independent boolean variable to rearrange and experiment with them during development. Still, due to the short-lived nature of callbacks, state variables are unavoidable and are actually the essence of object-oriented programming (i.e., *methods + mutable state*).

Similarly to Section 1.1, note also that the state machine is encoded across 3 different methods, each intermixing code with unrelated functionality.

Céu

The equivalent code for the *Bomber* action in Céu doesn't require state variables and reflects the sequential state machine implicitly, using `await` statements in direct style to separate the actions [9]:

```

1:  code/await Bomber (void) -> _ActionName__Enum
2:  do
3:      <...>
4:      spawn Mover();                // spawn the pingu movement to execute in the "background"
5:
6:      var&? Sprite s = spawn Sprite(<...>); // spawn the bomber animation to execute in the "background"
7:      watching s do
8:          // 1. 0th frame: plays a "Oh no!" sound.
9:          {Sound::PingusSound::play_sound("ohno", 0.5, 0.0)};
10:
11:         // 2. 10th frame: plays a "Bomb!" sound.
12:         await outer.game.dt until s!.sprite.frame == 10;
13:         {Sound::PingusSound::play_sound("plop", 0.5, 0.0)};
14:
15:         // 3. 13th frame: throws particles, destroys the terrain, shows an explosion sprite
16:         await outer.game.dt until s!.sprite.frame == 13;
17:         spawn PinguParticles(<...>) in outer.pingu_particles;
18:         call Game_Remove({&bomber_radius}, <...>);
19:         do
20:             <...>
21:             spawn Sprite(<...>);          // explosion
22:
23:             // 4. Game tick: hides the explosion sprite
24:             await outer.game.dt;
25:         end
26:         await FOREVER;
27:     end
28:
29:     // 5. Last frame: kills the pingu
30:     escape {ActionName::DEAD};
31: end

```

The *Bomber* is a `code/await` abstraction of Céu, which is similar to a [coroutine](#): a subroutine that retains runtime state, such as local variables and the program counter, across reactions to events (i.e., across `await` statements). The pingu movement and sprite animation are isolated in two other abstractions and execute in separate through the `spawn` primitive (ln. 4,6). The event `game.dt` (ln. 12,12,24) is analogous to the `update` callback of C++ and occurs on every frame.

The code tracks the animation instance (ln. 7-27), performing the last bomber action on termination (ln. 30). As soon as the animation starts, the code performs the first action (ln. 8). The intermediate actions are performed when the corresponding conditions occur (ln. 12,16,24). The `do-end` block (ln. 19-25), restricts the lifespan of the single-frame explosion sprite (ln. 21): after the next game tick (ln. 24), the block terminates and automatically destroys the spawned abstraction (removing it from the screen).

In contrast with the implementation in C++, all actions happen in a contiguous chunk of code (ln. 6-30) which handles no extra functionality.

Summary:

The structured constructs of Céu provide some advantages in comparison to explicit state machines:

- They encode all states with direct sequential code, eliminating shared state variables.
- They handle all states (and only them) in the same contiguous block, improving code encapsulation.

How common are Finite State Machines?

Pingus supports 16 actions in the game [9]: 5 of them implement at least one state machine and are considerable smaller in

Céu in terms of LoCs:

Action	Céu	C++	Explicit State
-----	----	----	-----
Bomber	23	50	4 state variables
Bridger	75	100	2 state variables
Drown	6	15	1 state variable
Exiter	7	22	2 state variables
Splashed	6	19	2 state variables

Considering the other 11 actions, the reduction in LoCs is negligible. This asymmetry in the implementation of actions illustrates the gains in expressiveness when describing state machines in direct style.

As other examples, detecting mouse dragging in the scenario [🔗] and in the small map [🔗] to move the game viewport also involves state machines. State machines also appear in the *FPS counter* [🔗] and in UI widgets with visual feedback (e.g., [🔗]).

Among the 65 implementation files in Céu, we found 29 cases in 25 files using structured mechanisms to substitute states machines. They manifest as `await` statements in sequence or in aborting constructs such as `par/or` and `watching`.

2) Continuation Passing

The completion of a long-lasting activity may carry a continuation, i.e., some action to execute next.

2.1) Advancing Pages in the Story Screen

The clickable *blue dots* in the campaign world map transit to ambience story screens (Figure 7). A story is composed of multiple pages and, inside each page, the words of the story appear incrementally over time. A first click in the button `>>>` fast forwards the words to show the full page. A second click advances to the next page, until the story terminates. If the page completes before a click (due to the time elapsing), a first click advances to the next page.

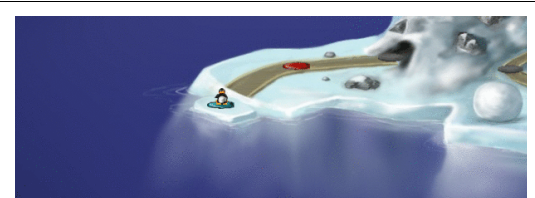


Figure 7: The Story screen

C++

In C++, the class `StoryScreenComponent` [🔗] implements the method `next_text`, which is a callback for clicks in `>>>`:

```

1: StoryScreenComponent::StoryScreenComponent (<...>) :
2:   <...>
3: {
4:   pages          = <...>;           // vector with loaded pages
5:   current_page   = pages.back();    // first loaded page
6:   displayed      = false;           // if current page is complete
7:   <...>
8: }
9:
10: <...>    // draw page over time
11:
12: void StoryScreenComponent::update (<...>) {
13:   <...>
14:   if (<all-words-appearing>) {
15:     displayed = true;
16:   }
17: }
18:
19: void StoryScreenComponent::next_text() {
20:   if (!displayed) {
21:     displayed = true;
22:     <...>           // remove current page
23:   } else {
24:     pages.pop_back();
25:     if (!pages.empty()) {           // next page
26:       current_page = pages.back();
27:       displayed    = false;
28:       <...>
29:     } else {
30:       <...>           // terminates the story screen
31:     }
32:   }
33: }
```

The variable `pages` (ln. 4-5, 24-26) is a vector holding each page, but which also encodes *continuations* for the story progress: each call to `next_text` that advances the story (ln. 23-32) removes the current page (ln. 24) and sets the next action to perform (i.e., "display a new page") in the variable `current_page` (ln. 26). Figure 8 illustrates the continuation mechanism to advance

pages and also a state machine for fast forwarding words (inside the dashed rectangle). The state variable `displayed` (ln. 6,15,20,21,27) switches between the behaviors "advancing text" and "advancing pages", which are both handled intermixed inside the method `next_text`.

Céu

The code in Céu [🔗] uses the [internal event](#) `next_text`, which is emitted from clicks in `>>>`:

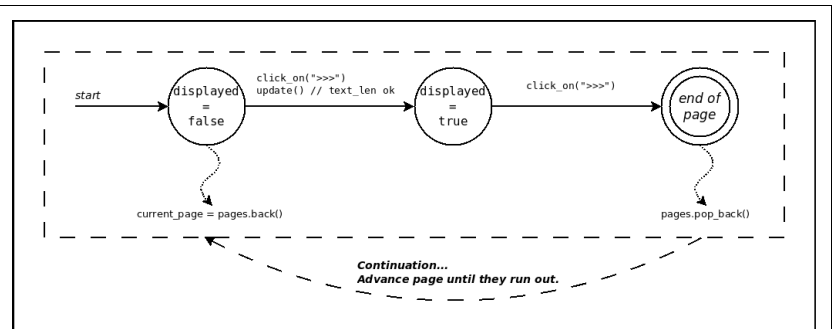


Figure 8: State machine for the *Story* screen

```

1: code/await Story (void) -> bool do
2:   <...>
3:   event void next_text; // emitted from clicks in '>>>'
4:
5:   { pages = <...>; } // same as in C++
6:   loop i in [0 <- {pages.size()}] do
7:     par/or do
8:       watching next_text do
9:         <...> // loop to advance text over time
10:        end
11:        await next_text;
12:      with
13:        <...> // loop to redraw current _pages[i]
14:      end
15:    end
16:  end

```

The sequential navigation from page to page uses a loop in direct style (ln. 6-15) instead of explicit state variables for the continuation and state machine. While the text advances in an inner loop (hidden in ln. 9), we watch the `next_text` event that fast forwards it. The loop may also eventually terminate with the time elapsing normally. This way, we do not need a variable (such as `displayed` in C++) to switch between the states "advancing text" and "advancing pages". The `par/or` makes the page advance logic to execute in parallel with the redrawing code (ln. 13). Whenever the page advances, the redrawing code is automatically aborted (due to the `or` modifier). The `await next_text` in sequence (ln. 11) is the condition to advance to the next page.

Note that, unlike the implementation in C++, the "advancing text" behavior is not intermixed with the "advancing pages" behavior, instead, it is encapsulated inside the inner loop nested with a deeper indentation (ln. 9).

2.2) Transition to the *Credits* Screen from the *Story* Screen

The world map has clickable blue dots for both introductory and ending stories. For introductory stories, the game returns to the world map after displaying the pages. For ending stories, the game also displays a *Credits* screen before returning to the world map (Figure 9).

C++

In C++, the class `StoryDot` [🔗] reads the level file to check whether the story should, after termination, display the *Credits* screen or not:

```

1: StoryDot::StoryDot(const FileReader& reader) :
2:   m_credits(false), // by default, do not display
3: {
4:   <...>
5:   reader.read_bool("credits", m_credits); // read from the file
6: }
7:
8: void StoryDot::on_click() {
9:   <...>
10:  SceneManager::instance()->push_screen(<StoryScreen>(<...>, m_credits));
11:   <...>
12: }

```

The boolean variable `m_credits` is passed to the class `StoryScreen` (ln. 10) [🔗] and represents the screen continuation, i.e., what to do after displaying the story. The `StoryScreen` then forwards the continuation [🔗] even further to the auxiliary class `StoryScreenComponent` [🔗] (presented in [Section 2.1](#)):

```

1: StoryScreenComponent::StoryScreenComponent (<...>) :
2:   m_credits(credits),
3:   <...>
4: {

```

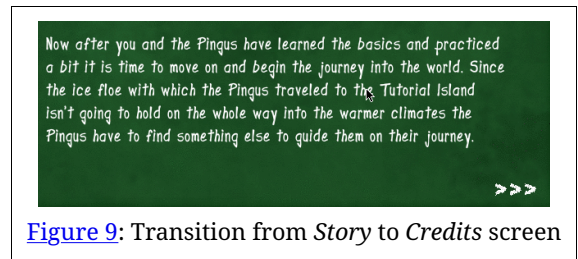


Figure 9: Transition from *Story* to *Credits* screen

```

5:     <...>
6: }
7:
8: <...>    // draw and update page
9:
10: void StoryScreenComponent::next_text() {
11:     if (!displayed) {
12:         <...>
13:     } else {
14:         <...>
15:         if (!pages.empty()) {
16:             <...>
17:         } else {
18:             if (m_credits) {
19:                 ScreenManager::instance()->replace_screen(<Credits>(<...>));
20:             } else {
21:                 ScreenManager::instance()->pop_screen();
22:             }
23:         }
24:     }
25: }

```

When the method `next_text` has no pages left to display (ln. 17-23), it decides where to go next, depending on the continuation flag `m_credits` (ln. 18).

Céu

In Céu, the flow between the screens to display is a direct sequence of statements [9]:

```

1: loop do
2:     var int ret = await Worldmap();
3:     if ret=={WORLDMAP_RETURN_STORY_MAP} or ret=={WORLDMAP_RETURN_STORY_CREDITS} then
4:         <...>
5:         var bool is_click = await Story();
6:         if is_click and ret=={WORLDMAP_RETURN_STORY_CREDITS} then
7:             <...>
8:             await Credits();
9:         end
10:    else
11:        <...>
12:    end
13: end

```

We first invoke the `Worldmap` (ln. 2), which exhibits the map and let the player interact with it until a dot is clicked. If the player selects a story dot (ln. 4-9), we invoke the `Story` and await its termination (ln. 5). Finally, we check the returned values (ln. 6) to perhaps display the `Credits` screen (ln. 8). The enclosing loop restores the `Worldmap` and repeats the process.

Discussion

Figure 10 illustrates the *continuation-passing style* of C++ and the *direct style* of Céu for screen transitions:

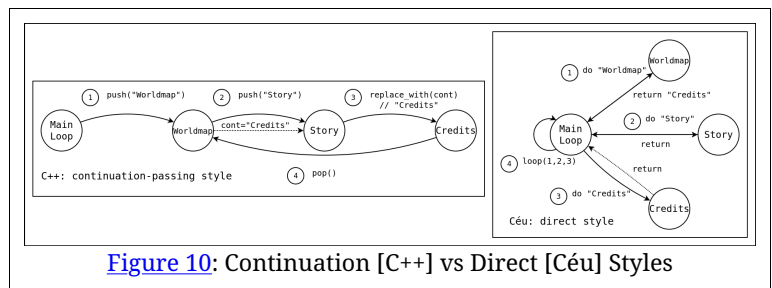


Figure 10: Continuation [C++] vs Direct [Céu] Styles

1. Main Loop => Worldmap:
 - C++ uses an explicit stack to push the `Worldmap` screen [9].
 - Céu invokes the `WorldMap` screen expecting a return value (ln. 2).
2. Worldmap (blue dot click) => Story:
 - C++ pushes the `Story` screen passing the continuation flag (`StoryDot`, ln. 10).
 - Céu stores the `Worldmap` return value and invokes the `Story` screen (ln. 2,5).
3. Story => Credits:
 - C++ replaces the current `Story` screen with the `Credits` screen (`StoryScreenComponent`, ln. 19).
 - Céu invokes the `Credits` screen after the `await Story` returns (ln. 8).
4. Credits => Worldmap:
 - C++ pops the `Credits` screen, going back to the `Worldmap` screen. Céu uses an enclosing loop to restart the process (ln. 1,13).

In contrast with C++, the screens in Céu are decoupled and only the `Main Loop` touches them: the `Worldmap` has no references to `Story`, which has no references to `Credits`.

Summary:

The direct style of Céu has some advantages in comparison to the continuation-passing style:

- It uses structured control flow (i.e., sequences and loops) instead of explicit data structures (e.g., stacks) or continuation variables.
- The activities are decoupled and do not hold references to one another.
- A single parent class describes the flow between the activities in a self-contained block of code.

How common is Continuation Passing?

Continuation passing typically controls the overall structure of the game, such as screen transitions in menus and level progressions.

Céu uses the direct style techniques in 5 cases involving screen transitions: the main menu [🎮], the level menu [🎮], the level set menu [🎮], the world map loop [🎮], and the gameplay loop [🎮]. It also uses the same technique for the loop that switches the pingu actions during gameplay [🎮].

3) Dispatching Hierarchies

Entities typically form a dispatching hierarchy in which a container that receives a stimulus automatically forwards it to its managed children.

3.1) Bomber draw and update callbacks

C++

In C++, the class Bomber [🎮] declares a sprite member to handle its animation frames (Figure 6):

```

1: class Bomber : public PinguAction {
2:     <...>
3:     Sprite sprite;
4: }
5:
6: Bomber::Bomber (<...>) : <...> {
7:     sprite.load(<...>);
8:     <...>
9: }
10:
11: void Bomber::update () {
12:     sprite.update ();
13: }
14:
15: void Bomber::draw (SceneContext& gc) {
16:     <...>
17:     gc.color().draw(sprite, <...>);
18: }
```

The Sprite class is part of the game engine and knows how to update [🎮] and render [🎮] itself. However, the Bomber still has to respond to update and draw requests from the game and forward them to the sprite (ln. 11-13 and 15-18).

To understand how the update callback flows from the original environment stimulus from the game down to the sprite, we need to follow a long chain of 7 method dispatches (Figure 11):

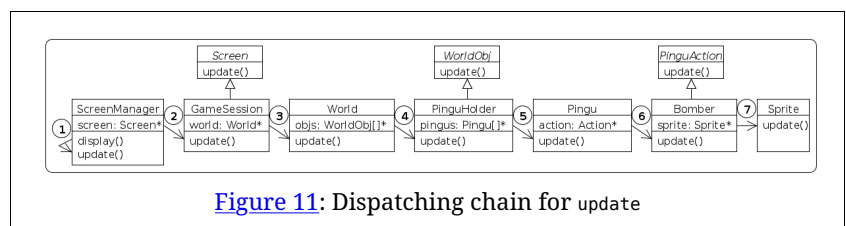


Figure 11: Dispatching chain for update

1. ScreenManager::display [🎮] in the main game loop calls update [🎮].
2. ScreenManager::update [🎮] calls last_screen->update [🎮] for the active game screen (a GameSession instance, considering the Bomber).
3. GameSession::update [🎮] calls world->update [🎮].
4. World::update [🎮] calls obj->update [🎮] for each object in the world.
5. PinguHolder::update [🎮] calls pingu->update [🎮] for each pingu alive.
6. Pingu::update [🎮] calls action->update [🎮] for the active pingu action.
7. Bomber::update [🎮] calls sprite.update [🎮]. Sprite::update [🎮] finally updates the animation frame.

Each dispatching step in the chain is necessary considering the game architecture:

- With a single assignment to last_screen, we can easily deactivate the current screen and redirect all dispatches to a new screen.
- The World class manages and dispatches events to all game entities, such as all pingus and traps, with a the common

```
interface WorldObj [🔗].
```

- Since it is common to iterate only over the pingus (vs. all world objects), the container `PinguHolder` manages all pingus.
- Since a single pingu can change between actions during lifetime, the `action` member decouples them with another level of indirection.
- Sprites are part of the game engine and are reusable everywhere (e.g., UI buttons, world objects, etc.), so it is also convenient to decouple them from actions.

The `draw` callback flows through the same dispatching hierarchy until reaching the `Sprite` class.

Céu

In Céu, the `Bomber` action spawns a `Sprite` animation instance on its body [🔗]:

```
1: code/await Bomber (void) -> _ActionName__Enum do
2:   <...>
3:   var&? Sprite sprite = spawn Sprite(<...>);
4:   <...>
5: end
```

The `Sprite` instance (ln. 3) can react directly to external `dt` [🔗] and `redraw` [🔗] events (which are analogous to `update` and `redraw` callbacks, respectively), bypassing the program hierarchy entirely. While and *only while* the bomber abstraction is alive, the sprite animation is also alive. The radical decoupling between the program hierarchy and reactions to events eliminates dispatching chains entirely.

Summary:

Passive entities subjected to hierarchies require a dispatching architecture that makes the reasoning about the program harder:

- The full dispatching chain may go through dozens of files.
- The dispatching chain may interleave between classes specific to the game and also classes from the game engine (possibly third-party classes).

How common are Dispatching Hierarchies?

In C++, the update subsystem touches 39 files with around 100 lines of code just to forward `update` methods through the dispatching hierarchy (e.g., class `GroupComponent` [🔗]). For the drawing subsystem, 50 files with around 300 lines of code (e.g., class `ArmageddonButton` [🔗]). The implementation in C++ also relies on dispatching hierarchy for `resize` callbacks, touching 12 files with around 100 lines of code (e.g., class `StartScreen` [🔗]).

Most of this code is eliminated in Céu since abstractions can react directly to the environment, not depending on hierarchies spread across multiple files.

Note that dispatching hierarchies cross game engine code, suggesting that most games use this control-flow pattern heavily. In the case of the Pingus engine, we rewrote 9 files from C++ to Céu, reducing them from 515 to 173 LoCs, mostly due to dispatching code removal.

4) Lifespan Hierarchies

Entities typically form a lifespan hierarchy in which a terminating container entity automatically destroys its managed children.

4.1) Game UI Widgets

C++

In C++, the class `GameSession` coordinates the UI widgets, such as the buttons, pingus counter, and small map (Figure 13) to coexist with the game screen during its whole lifespan [🔗]:

```
1: GameSession::GameSession(<...>) :
2:   <...>
3: {
4:   <...>
```

```

5:   button_panel = new ButtonPanel(<...>);      // these widgets are always active...
6:   pcounter     = new PingusCounter(<...>);
7:   small_map    = new SmallMap(<...>);
8:   <...>
9:   gui_manager->add(button_panel);             // ...but are added
10:  gui_manager->add(pcounter);                  // dynamically to the
11:  gui_manager->add(small_map);                 // dispatching hierarchy
12:  <...>
13: }

```



Figure 13: UI children with static lifespan

The widgets are created in the constructor (ln. 5-7), added to a UI container (ln. 9-11), and are never removed since they must always be visible. Arguably, to better express the intent of making them coexist with the game screen, the widgets could be declared as top-level automatic (non-dynamic) members. However, the class uses a container to automate *draw* and *update* dispatching to the widgets, as discussed in [Section 3.1](#). In turn, the container method *add* expects dynamically allocated children only because they are automatically deallocated inside the container destructor [9].

The dynamic nature of containers in C++ demand extra caution from programmers:

- When containers are part of a dispatching chain, it gets even harder to know which objects are dispatched: one has to "simulate" the program execution and track calls to *add* and *remove*.
- For objects with dynamic lifespan, calls to *add* must always have matching calls to *remove*: missing calls to *remove* lead to memory and CPU leaks (to be discussed as the *lapsed listener* problem further).

Céu

In Céu, entities that coexist just have to be created in the same lexical block:

```

1: code/await Game (void) do
2:   <...>           // other coexisting functionality
3:   spawn ButtonPanel(<...>);
4:   spawn PingusCounter(<...>);
5:   spawn SmallMap(<...>);
6:   <...>           // other coexisting functionality
7: end

```

Since abstractions can react independently, they do not require a dispatching container.

Lexical lifespan never requires containers, allocation and deallocation, or explicit references. In addition, all required memory is known at compile time, similarly to stack-allocated local variables.

Note that the actual code in the repository [9] is equivalent to the code using abstraction above, but inlines all functionality in parallel since they are instantiated only once:

```

1: par do
2:   <...>           // other coexisting functionality
3:   with
4:     #include "button_panel.ceu" // inlined code for the pannel
5:   with
6:     #include "pingus_counter.ceu" // inlined code for the counter
7:   with
8:     #include "small_map.ceu" // inlined code for the map
9:   with
10:    <...>          // other coexisting functionality
11: end

```

The [Bomber state machine](#) of [Section 1.2](#) also relies on lexical scope to delimit the lifespan of the explosion sprite to a single frame.

4.2) The Pingus Container

A pingu is a dynamic entity created periodically and destroyed under certain conditions, such as falling from a high altitude [9] (Figure 14).

C++

In C++, the class `PinguHolder` is a container that holds all pingus alive:

```

1: Pingu* PinguHolder::create_pingu (<...>) {
2:   <...>
3:   Pingu* pingu = new Pingu (<...>);
4:   pingus.push_back(pingu);
5:   <...>

```

```

6: }
7:
8: void PinguHolder::update() {
9:     <...>
10:    while(pingu != pingus.end()) {
11:        (*pingu)->update();
12:        if ((*pingu)->get_status() == Pingu::PS_DEAD) {
13:            pingu = pingus.erase(pingu);
14:        }
15:        <...>
16:        ++pingu;
17:    }
18: }

```

The method `PinguHolder::create_pingu` (ln. 1-6) is called periodically to create a new `Pingu` and add it to the `pingus` collection (ln. 3-4). The method `PinguHolder::update` (ln. 8-18) checks the state of all `pingus` on every frame, removing those with the dead status (ln. 12-14).

Entities with dynamic lifespan in C++ require explicit add and remove calls associated to a container (ln. 4,13). Without the `erase` call above, a dead pingu would remain in the collection with updates on every frame (ln. 11). Since the `redraw` behavior for a dead pingu is innocuous, the death could go unnoticed but the program would keep consuming memory and CPU time. This problem is known as the *lapsed listener* [9] and also occurs in languages with garbage collection: A container typically holds a strong reference to a child (sometimes the only reference to it), and the runtime cannot magically detect it as garbage.

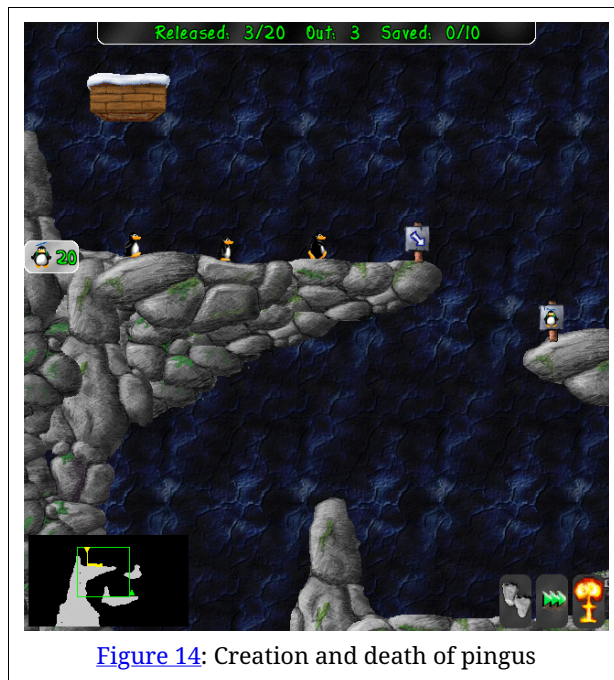


Figure 14: Creation and death of pingus

Céu

Céu supports `pool` declarations to hold dynamic abstraction instances. Additionally, the `spawn` statement supports a `pool` identifier to associate the new instance with a pool.

The game screen spawns a new `Pingu` on every invocation of `Pingu_Spawn` [9]:

```

1: code/await Game (void) do
2:     <...>
3:     pool[] Pingu pingus;
4:     code/await Pingu_Spawn (<...>) do
5:         <...>
6:         spawn Pingu(<...>) in outer.pingus;
7:     end
8:     <...> // code invoking Pingu_Spawn
9: end

```

The `spawn` statement (ln. 6) specifies the pool declared at the top-level block of the game screen (ln. 3). In this case, the lifespan of the new instances follows the scope of the pool (ln. 1-9) instead of the `spawn` enclosing scope (ln. 4-7). Since pools are also subject to lexical scope, the lifespan of all dynamically allocated `pingus` is constrained to the game screen.

Lexical scopes handle memory and event dispatching automatically for static instances and also for pools. However, the lifespan of a dynamic instance does not necessarily have to match the lifespan of its associated pool (Figure 15). In Céu, when the execution block of a dynamic instance terminates, which characterizes its *natural termination*, the instance is automatically removed from its pool. Therefore, dynamic instances do not require any extra bookkeeping related to containers or explicit deallocation.

To remove a pingu from the game in Céu, we just need to terminate its execution block according to the appropriate conditions [9]:

```

1: code/await Pingu (<...>) do
2:     <...>
3:     loop do
4:         await outer.game.dt;
5:         if call Pingu_Rel_Getpixel(0,-1) == {Groundtype::GP_OUTOFSCREEN} then
6:             <...>
7:             escape {PS_DEAD};
8:         end
9:     end
10: end

```

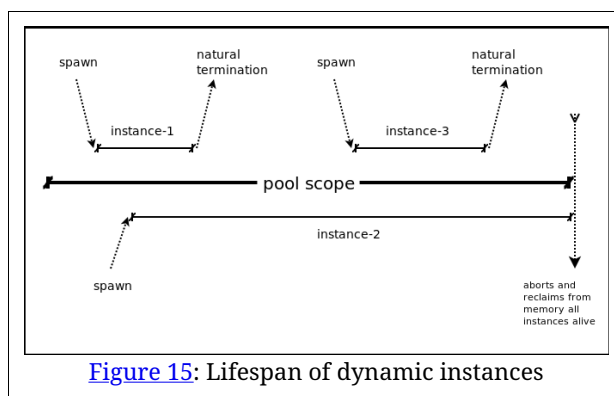


Figure 15: Lifespan of dynamic instances

The `escape` statement (ln. 7) aborts the execution block of the `Pingu` instance, removing it from its associated pool automatically. Hence, a dynamic instance that terminates naturally leaves no traces in the program.

Summary:

Lexical lifespan for static instances and natural termination for dynamic instances provide some advantages in comparison to lifespan hierarchies through containers:

- Lexical scope makes an abstraction lifespan explicit in the source code.
- The memory for static instances is known at compile time.
- Natural termination makes an instance innocuous and, hence, susceptible to immediate reclamation.
- Abstraction instances (static or dynamic) never require explicit manipulation of pointers/references.

How common are Lifespan Hierarchies?

All entities in a game have an associated lifespan.

The implementation in Céu has over 200 static instantiations spread across all 65 files. For dynamic entities, it defines 23 pools in 10 files, with almost 96 instantiations across 37 files. Pools are used to hold explosion particles, levels and level sets from files, gameplay & worldmap objects, and UI widgets.

5) Signaling Mechanisms

Entities often need to communicate explicitly through signaling mechanisms, especially if there is no hierarchy relationship between them.

5.1) Pausing the World

A click in the *Pause* button at the bottom right of the screen pauses all world objects, such as the clouds and pingus, but not other elements, such as the *Armageddon* button animation (Figure 16). The *Pause* button is also affected when the player presses `P` on the keyboard and indicates its state with dark and light backgrounds.

C++

In C++, the class `PauseButton` [🔗] handles mouse clicks to toggle the world pause state which is checked when redrawing the button:

```

1:  PauseButton::PauseButton(GameSession s, <...>):
2:      RectComponent(<...>),
3:      session(s),
4:      background("core/buttons/hbuttonbgb"),
5:      backgroundhl("core/buttons/hbuttonbg"),
6:      <...>
7:  {
8:      <...>
9:  }
10:
11: void PauseButton::on_click (<...>) {
12:     session->set_pause(!session->get_pause());
13: }
14:
15: void PauseButton::draw (<...>) {
16:     <...>
17:     if (session->get_pause()) {
18:         gc.draw(backgroundhl, <...>);
19:     } else {
20:         gc.draw(background, <...>);
21:     }
22:     <...>
23: }
```

The mouse callback `on_click` (ln. 11-13) toggles the pause state. Depending on the current state, the method `draw` (ln. 15-23) chooses the appropriate background sprite loaded in the class constructor (ln. 4-5).

The class `GameSession` [🔗] handles keyboard presses and applies the pause state to the world:

```

1: void GameSession::on_pause_press () {
```

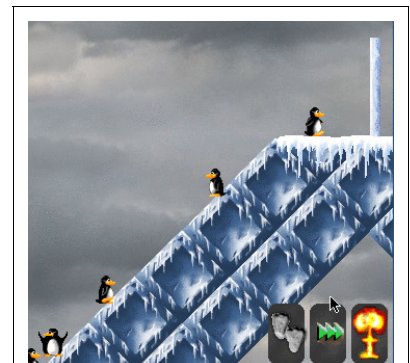


Figure 16: Pausing the world.

```

2:     set_pause(!get_pause());
3: }
4:
5: void GameSession::update_server (<...>) {
6:     <...>
7:     if (!get_pause()) {
8:         <...>
9:         server->update();
10:    }
11:    <...>
12: }

```

The call to the world `update` (ln. 9) only applies if the world is not paused (ln. 7). Since `update` methods propagate throughout the world hierarchy, skipping the call to the root method effectively pauses the world.

Céu

As discussed in [Section 3.1](#), entities in Céu do not update through dispatching hierarchies, but instead react directly to events. This way, the pausing technique applied in C++ is not effective in Céu.

In Céu, the *Pause* button communicates with the rest of the application through the event `go_pause_toggle` [🔗]:

```

1: <...>
2: var& RectComponent c = spawn RectComponent(<...>);
3: spawn do
4:     loop do
5:         watching go_pause_toggle do
6:             spawn Sprite(<...>, "core/buttons/hbuttonbgb");
7:             await c.component.on_click;
8:             emit go_pause_toggle;
9:         end
10:        watching go_pause_toggle do
11:            spawn Sprite(<...>, "core/buttons/hbuttonbgb");
12:            await c.component.on_click;
13:            emit go_pause_toggle;
14:        end
15:    end
16: end
17: <...>

```

The button toggles between showing the dark (ln. 10-14) and light (ln. 5-9) background sprites based on their lexical scope. The background changes when the button is clicked (ln. 7,12) or when `go_pause_toggle` is emitted from a keyboard press (ln. 5,10). The button also broadcasts `go_pause_toggle` whenever it is clicked (ln. 8,13).

The pausing mechanism relies on two update events, `game.dt` for the game world, and `main.dt` for the rest of the application [🔗]:

```

1: event void go_pause_toggle;
2: <...>
3: spawn do
4:     var bool is_paused = false;
5:     par do
6:         every go_pause_toggle do
7:             is_paused = not is_paused;
8:         end
9:         with
10:            every outer.main.dt do
11:                <...>
12:                if not is_paused then
13:                    emit outer.game.dt(<...>);
14:                end
15:            end
16:        <...>
17:    end
18: end
19: <...>

```

Whenever `go_pause_toggle` is emitted, the local state variable `is_paused` is toggled (ln. 6-8). Also, whenever `main.dt` occurs (ln. 10-15), the event `game.dt` is emitted only if the world is not paused (ln. 12-14).

World entities are set to react to `game.dt`, while all other entities are set to react to `main.dt`. Since all world entities are `Sprite` instances, the abstraction interface receives its update event as a reference [🔗]. On creation, world and non-world sprites pass distinct events, e.g.:

- The *Bomber* action uses `game.dt` [🔗], since it is a world entity.
- The *Armageddon* button uses `main.dt` [🔗], since it should not pause together with the world entities.

This technique contrasts with the implementation in C++, which prevents the `update` dispatching chain to flow from the world "root" towards the sprite "leaves". In Céu, the sprite "leaves" execute detached from a hierarchy, but do not update

because the event of interest is never generated in the paused state.

5.2) Global Keys and the Options Menu

The *Mouse Grab* option restricts the mouse movement to the game window boundaries (Figure 17). The option can be set anywhere in the game by pressing *Ctrl-G*. In addition, the *Options* menu has a check box to toggle the *Mouse Grab* option with mouse clicks while still responding to *Ctrl-G* presses.

The implementations in C++ and Céu use a signalling mechanism to connect the key presses, the check box, and a configuration manager that applies the appropriate side effects in the game (i.e., restrict the mouse movement). Figure 18 illustrates how the mutual notifications create a dependency cycle between the configuration manager and the check box.

C++

In C++, the class `GlobalEvent` [🔗] detects *Ctrl-G* presses and invokes the callback `config_manager.set_mouse_grab`:

```
1: void GlobalEvent::on_button_press (<...>) {
2:   <...>
3:   switch (event.keysym.sym) {
4:     case SDLK_g:
5:       if (keystate[SDLK_LCTRL] || keystate[SDLK_RCTRL]) {
6:         config_manager.set_mouse_grab(
7:           !config_manager.get_mouse_grab());
8:       }
9:       break;
10:    <...>
11:  }
12: }
```

The class `ConfigManager` [🔗] uses a `boost::signal` [🔗] to notify the application when the new configuration is applied:

```
1: boost::signals2::signal<void(bool)> on_mouse_grab_change; // definition in `config_manager.h`
2:
3: void ConfigManager::set_mouse_grab (bool v) {
4:   <...>
5:   if (v != get_mouse_grab()) {
6:     <...> // the actual "grab" effect
7:     on_mouse_grab_change(v);
8:   }
9: }
```

The `if` enclosing the signal emission (ln. 5-8) breaks the dependency cycle of Figure 18 and prevents an infinite execution loop.

The class `CheckBox` [🔗] also uses a `boost::signal` to notify the application on changes:

```
1: boost::signals2::signal<void (bool)> on_change; // definition in `check_box.hpp`
2:
3: void CheckBox::set_state (bool is_on, bool send_signal) {
4:   <...> // switches the check box state
5:   if (send_signal) {
6:     on_change(is_on);
7:   }
8: }
```

Again, the `if` enclosing the signal emission (ln. 5-7) breaks the dependency cycle of Figure 18 to avoid infinite execution.

The class `OptionsMenu` [🔗] creates the dependency loop by connecting the two signals:

```
1: typedef std::vector<boost::signals2::connection> Connections; // definition in `option_menu.hpp`
2: Connections connections; // definition in `option_menu.hpp`
3:
4: OptionMenu::OptionMenu() :
5:   connections(),
6:   mousegrab_box(),
7:   <...>
8: {
9:   mousegrab_box = new CheckBox(<...>);
```



Figure 17: The *Mouse Grab* configuration option.

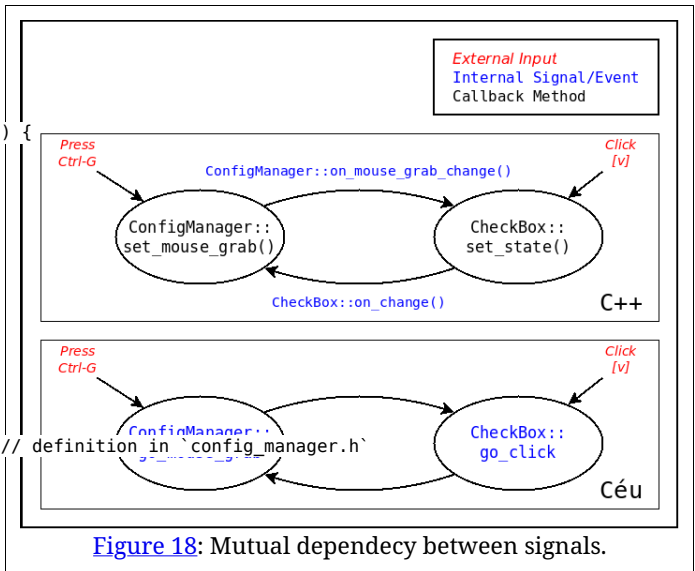


Figure 18: Mutual dependency between signals.

```

10:     connections.push_back(
11:         config_manager.on_mouse_grab_change.connect(
12:             std::bind(&CheckBox::set_state, mousegrab_box, <...>, false);
13:         )
14:     );
15:     connections.push_back(
16:         mousegrab_box->on_change.connect(
17:             std::bind(&ConfigManager::set_mouse_grab, &config_manager, <...>);
18:         )
19:     );
20:     <...>
21:
22: }
23:
24: OptionMenu::~~OptionMenu() {
25:     for (Connections::iterator i=connections.begin(); i!=connections.end(); ++i) {
26:         (*i).disconnect();
27:     }
28: }

```

The constructor binds the signal `config_manager.on_mouse_grab_change` to the callback method `mousegrab_box->set_state` (ln. 10-14), and also the signal `mousegrab_box->on_change` to the callback method `config_manager.set_mouse_grab` (ln. 15-19). This way, every time the `ConfigManager` signals `on_mouse_grab_change` (`ConfigManager`, ln. 7 [up](#)), `set_state` is implicitly called. The same happens between the signal `on_change` in the `CheckBox` and the method `set_mouse_grab` in the `ConfigManager` (`ConfigManager`, ln. 3 [up](#)).

Note that the signal binding to call `CheckBox::set_state` (ln. 12) receives a fixed value `false` as the last argument to prevent infinite execution (`CheckBox`, ln. 3 [up](#)).

The destructor (ln. 24-28) breaks the connections explicitly when the *Option* screen terminates.

Céu

In Céu, a *Ctrl-G* key press broadcasts the internal event `config_manager.go_mouse_grab` to the application [🔗]:

```

1: spawn do
2:     var _SDL_KeyboardEvent&& e;
3:     every e in SDL_KEYDOWN do
4:         var _u8&& keystate = _SDL_GetKeyState(null);
5:         <...>
6:         if e:keysym.sym == {SDLK_g} then
7:             if ((keystate[{SDLK_LCTRL}] as bool) or (keystate[{SDLK_RCTRL}] as bool)) then
8:                 emit config_manager.go_mouse_grab(
9:                     not ({config_manager.get_mouse_grab()} as bool));
10:             end
11:         end
12:     <...>
13: end
14: end

```

The configuration manager [🔗] just needs to react to `go_mouse_grab` continuously to perform the *grab* effect:

```

1: data ConfigManager with
2:     event bool go_mouse_grab;
3: end
4: var ConfigManager config_manager = val ConfigManager(_);
5:
6: spawn do
7:     var bool v;
8:     every v in config_manager.go_mouse_grab do
9:         <...> // the actual "grab" effect
10:     end
11: end

```

The `CheckBox` [🔗] exposes the event `go_click` for notifications in both directions, i.e., from the abstraction to the application and *vice versa*:

```

1: data ICheckBox with
2:     var bool is_on;
3:     event bool go_click;
4: end
5:
6: code/await CheckBox (<...>) -> (var ICheckBox checkbox) -> FOREVER do
7:     checkbox = val ICheckBox(<...>);
8:     <...>
9:     par do
10:         every c.component.on_click do
11:             emit checkbox.go_click(not checkbox.is_on);
12:         end
13:     with
14:         loop do
15:             <...> // switches the check box state
16:             checkbox.is_on = await checkbox.go_click;
17:         end

```

```

18:     end
19: end

```

The abstraction reacts to external clicks continuously (ln. 10-12) to broadcast the event `go_click` (ln. 11). It also reacts continuously to `go_click` in another line of execution (ln. 14-17), which awakes from notifications from the first line of execution or from the application.

The `OptionsMenu` [9] connects the two events as follows:

```

1:  code/await OptionMenu <...> do
2:    <...>
3:    var& CheckBox b2 = <...>;
4:    spawn do
5:      par do
6:        var bool v;
7:        every v in outer.config_manager.go_mouse_grab do
8:          emit b2.checkbox.go_click(v);
9:        end
10:       with
11:         var bool v;
12:         every v in b2.checkbox.go_click do
13:           emit outer.config_manager.go_mouse_grab(v);
14:         end
15:       end
16:     end
17:   <...>
18: end

```

The two loops in parallel handle the connections in opposite directions: from the configuration manager to the check box (ln. 7-9); and from the check box to the configuration manager (ln. 12-14).

When the *Option* screen terminates, the connections break automatically since the body is automatically aborted.

Note that the implementation in Céu does not check event emits to break the dependency cycle and prevent infinite execution. Due to the [stack-based execution for internal events](#) in Céu, programs with mutually-dependent events do not create infinite execution loops.

Summary:

First-class internal events of Céu provide some advantages in comparison to Boost signals:

- They use the same convenient syntax of external events (e.g., `emit`, `await`, `every`, etc.).
- They never create infinite dependency loops.
- They do not require explicit unbinding.

How common are Signalling Mechanisms?

The implementation in Céu uses 39 events for internal communication defined in 23 files with over 200 invocations of `emit` and `await` spread in over 50 files.

Internal events are used for resizing entities, broadcasting update and draw requests, pausing parts of the game, triggering new pingu actions, signalling collisions, signalling UI interactions, among many others cases.

Conclusion

We promote the *structured synchronous reactive* programming model of Céu for the development of games. We present 9 in-depth use cases applied to *Pingus* (an open-source *Lemmings* clone), categorized in 5 control-flow patterns that likely apply to other games.

We show how the standard way to program games with objects and callbacks in C++ hinders structured programming techniques, such as support for sequential execution, long-lasting loops, and persisting local variables. In this sense, callbacks actually disrupt structured programming, becoming "[our generation's goto](#)" according to Miguel de Icaza.

Overall, we believe that most difficulties in implementing control behavior in game logic is not inherent to this domain, but a result of accidental complexity due to the lack of structured abstractions and an appropriate concurrency model to handle event-based applications.

Author

Francisco Sant'Anna

- <http://www.ceu-lang.org/chico/>
- <https://github.com/fsantanna/>
- [@_fsantanna](#)

Acknowledgments

[Leonardo Kaplan](#) and [Alexander Tkachov](#) for early explorations and prototypes.